

A young child wearing a brown aviator hat and goggles is sitting on the shoulder of a large, white, humanoid robot. The child is pointing their right index finger towards a large, stylized globe in the background. The robot's head is turned towards the child, and its right arm is raised, with the hand near its head. The background is a light blue sky with several white streaks resembling meteors or shooting stars. The globe is a light yellow color with a grid pattern.

昇思+昇腾开发板：minGPT开发、 训练及推理全流程实践

昇腾开发板 - OrangePi Alpro



香橙派OrangePi Alpro采用昇腾AI技术路线，具体为4核64位处理器+AI处理器，集成图形处理器，支持8/20 TOPS AI算力，广泛适用于AI边缘计算、深度视觉学习及视频流AI分析等各个行业。满足大多数AI算法原型验证、推理应用开发的需求。

香橙派开发板相关资料的获取方式

官方资料



外壳及散热器安装资料

下载



官方工具

下载



用户手册

下载



原理图

下载



机械图

下载



linux源码

下载

官方镜像



ubuntu镜像

下载



openEuler

openeuler镜像

下载

样例丰富，易学易用

香橙派论坛(<http://forum.orangepi.cn/>)、昇腾社区(hiascend.com)、昇思社区(mindspore.cn)为开发者免费提供数百个代码参考样例。



香橙派官方资料



昇腾开发板专区



昇思开发板开发教程

昇思MindSpore+ 香橙派技术能力进展



orange-pi-mindspore案例代码仓

- **开发友好：**动态图易用性提升，类huggingface风格降低开发调试门槛

```
prompts = [
    "Simply put, the theory of relativity states that ",
    "My favorite all time favorite condiment is ketchup.",
]
```

```
NUM_TOKENS_TO_GENERATE = 40
```

```
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = LlamaTokenizer.from_pretrained(model_id)
model = LlamaForCausalLM.from_pretrained(model_id, ms_dtype=mindspore.float16)
```

```
# quantize_cfg = w8x8(model.model.config)
# quantize(model, cfg=quantize_cfg)
```

```
model.jit()

inputs = tokenizer(prompts, return_tensors="ms", padding=True)
set_pyboost(False)

@mmindspore.jit(jit_config=mindspore.JitConfig(jit_syntax_level='STRICT'))
def decode_one_tokens(model, cur_token, input_pos, cache_position, past_key_values):
    logits = model(
        cur_token,
        position_ids=input_pos,
        cache_position=cache_position,
        past_key_values=past_key_values,
        return_dict=False,
        use_cache=True
    )[0]
    new_token = ops.argmax(logits[:, -1], dim=-1)[:, None]
    return new_token
```

- **性能提升：**mindspore.jit编译成图，三行代码实现推理性能提升一倍

- **全流程支持：**在香橙派上支持模型训推

支持的模型清单

【推理】

- DeepSeek-R1-Distill-Qwen-1.5B
- DeepSeek-Janus-Pro-1B
- Qwen1.5-0.5b
- TinyLlama-1.1B
- MiniCPM3-4B
- DctNet
- ResNet50
- ViT
- FCN
- ShuffleNet
- SSD
- RNN
- LSTM+CRF
- GAN
- DCGAN
- Pix2Pix
- Diffusion
- ResNet50_transfer

【训练】

- DeepSeek-R1-Distill-Qwen-1.5B
- BERT
- minGPT

目录

01 minGPT介绍

02 Attention机制实现

03 GPT架构

04 解码原理

05 混合精度

minGPT介绍

available GPT implementations



minGPT



本项目是将Karpathy的minGPT项目迁移至MindSpore框架的实现，包括训练和推理全流程。由于目前大多数可用的GPT模型实现都略显杂乱，minGPT力求小巧、简洁、易懂且具有教育意义。GPT并不是一个复杂的模型，此实现大约有 300 行代码

代码结构

```
02-minGPT/
├── mingpt/
│   ├── __init__.py # (package)标志性文件
│   ├── model.py    # 模型文件，包括attention、transformer_block以及GPT实现，需要开发者完成代码补充
│   ├── trainer.py  # 训练器文件，设置了o2模式的混合精度训练，需要开发者完成代码补充
│   └── utils.py    # 工具箱文件
├── demo.py         # GPT-nano数字排序训推全流程
└── run_demo.sh     # 脚本文件，用于自动拉起demo.py的训推
```

minGPT库目前由4个文件组成：其中mingpt/model.py包含实际的Transformer模型定义；mingpt/trainer.py是基于MindSpore框架的通用模型训练代码，用于在香橙派上的混合精度模型训练；mingpt/utils.py为项目工具文件。

此外，demo.py文件展示了在香橙派环境中对GPT和Trainer进行的极简使用，示例内容为简单的排序操作，包含从生成数据集→实例化模型→模型训练→模型验证全流程，开发者可在该文件中看到更加具体的相关示例。

目录

01 minGPT介绍

02 Attention机制实现

03 GPT架构

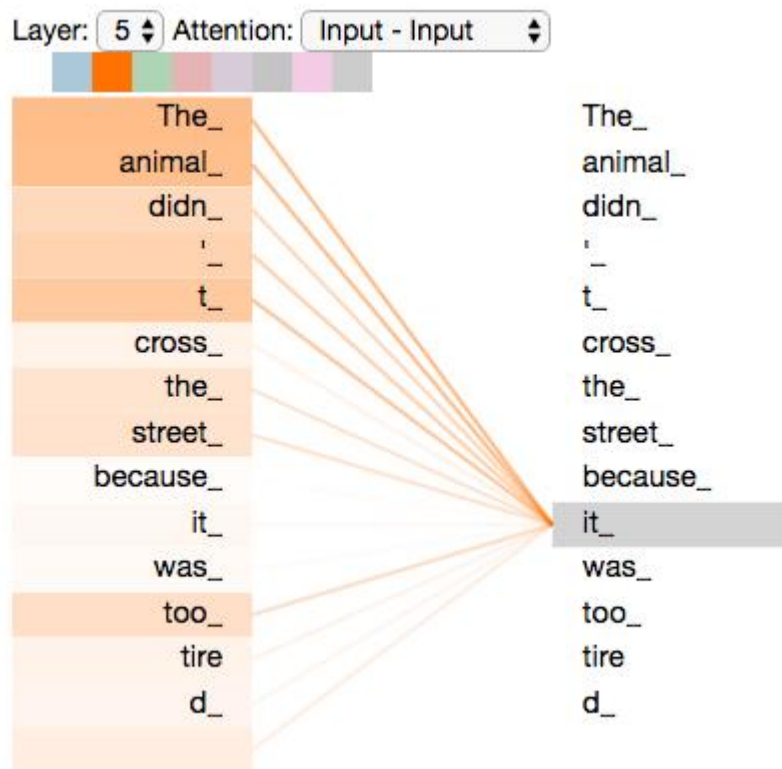
04 解码原理

05 混合精度

自注意力机制

The animal didn't cross the street because it was **too** tired.

这句话中的“它”指的是什么？它指的是街道还是动物？



自注意力机制

input

流浪

地球

Embedding

x_1



x_2



Query

q_1

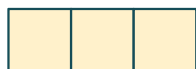


q_2

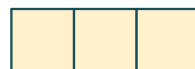


Key

k_1



k_2

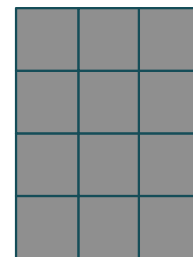


Value

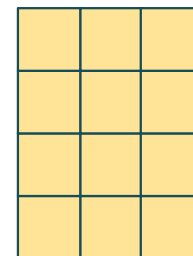
v_1



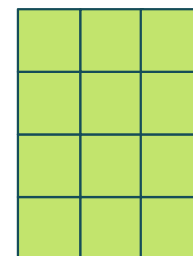
v_2



W^Q

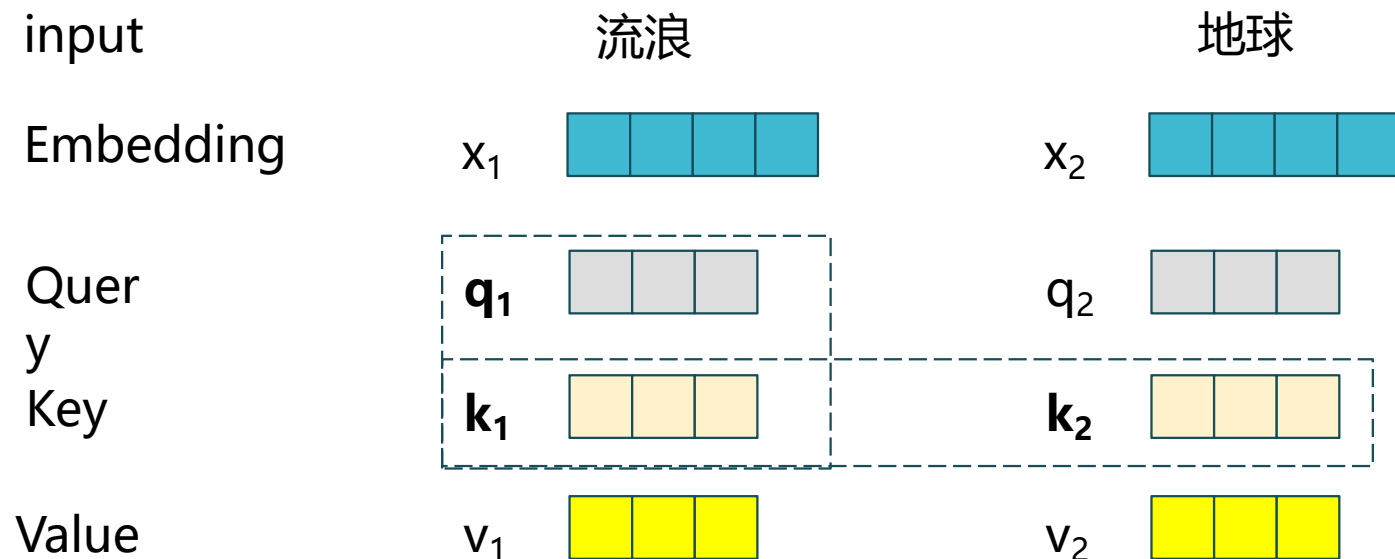


W^K



W^V

自注意力机制

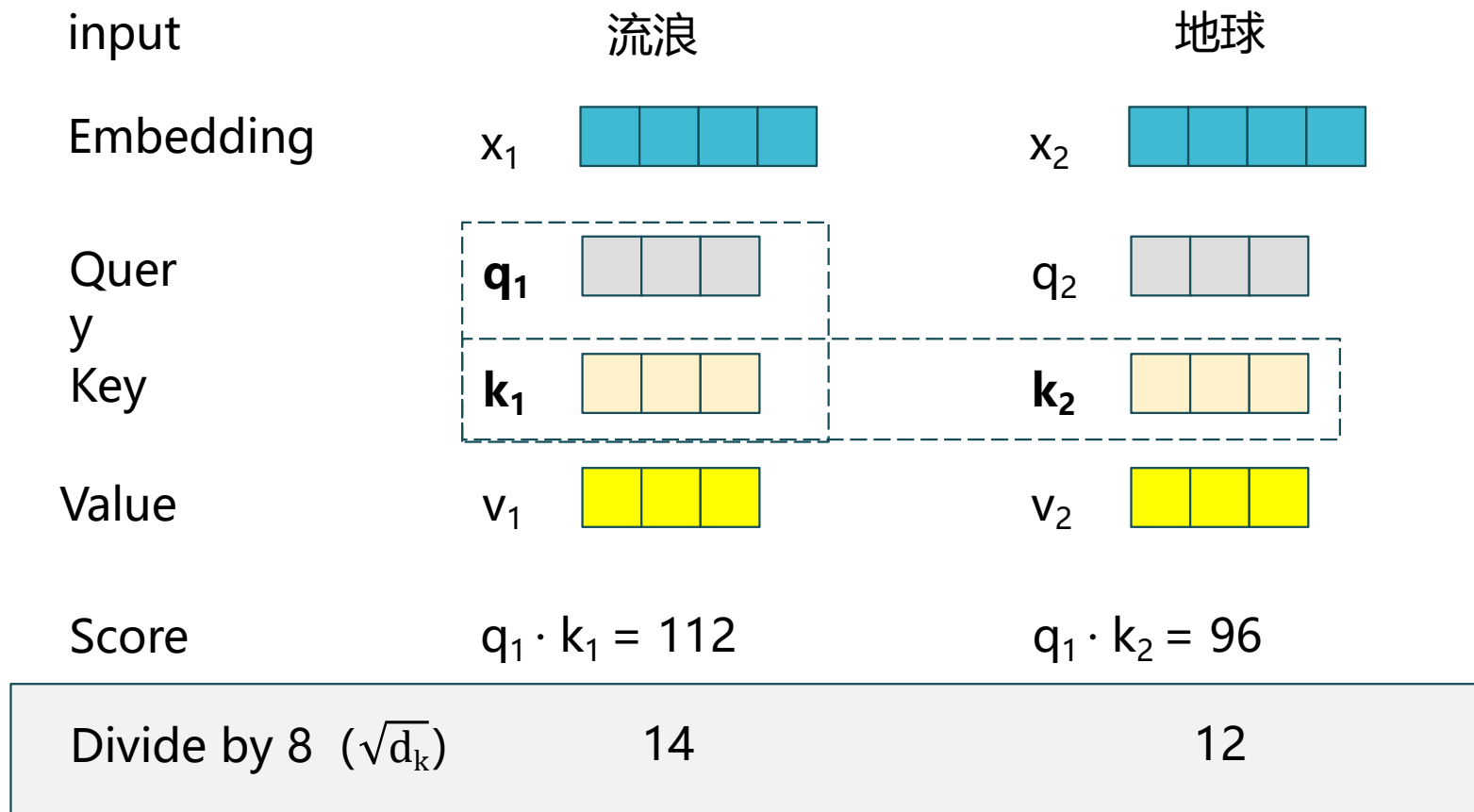


Score

$$q_1 \cdot k_1 = 112$$

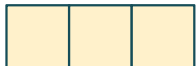
$$q_1 \cdot k_2 = 96$$

自注意力机制

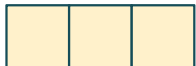


有些qk对在经过点积运算后的结果相对于其他点积对来说过大，这就导致在之后的softmax中占据了绝大多数注意力，而其他的点积对在经过softmax之后的分数则趋于0，结果就是会出现梯度消失的问题。因此这里除以一个常数的操作类似于归一化，使得各点积对的结果更加平滑。

自注意力机制

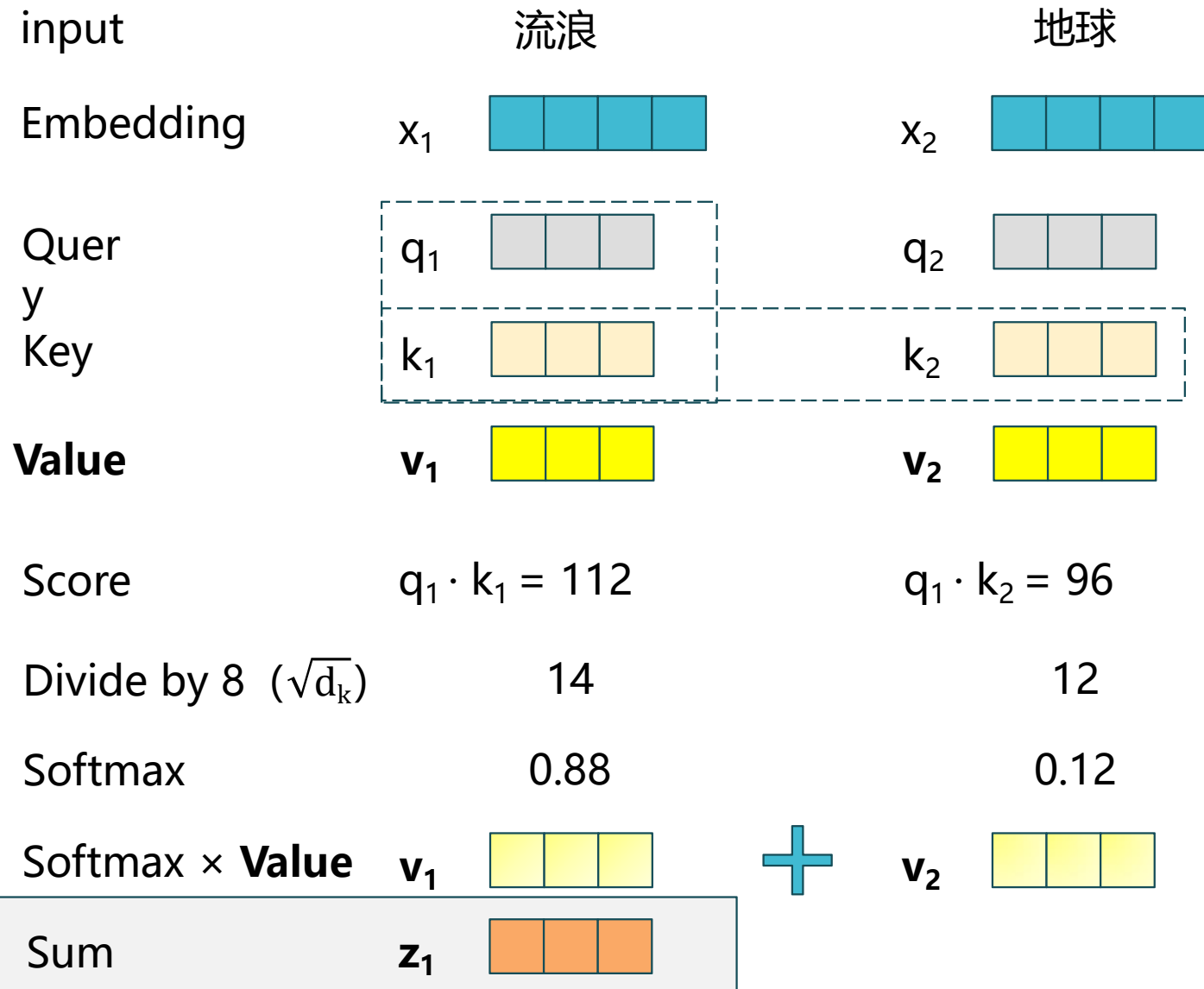
input	流浪	地球
Embedding	x_1 	x_2 
Query	q_1 	q_2 
Key	k_1 	k_2 
Value	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

自注意力机制

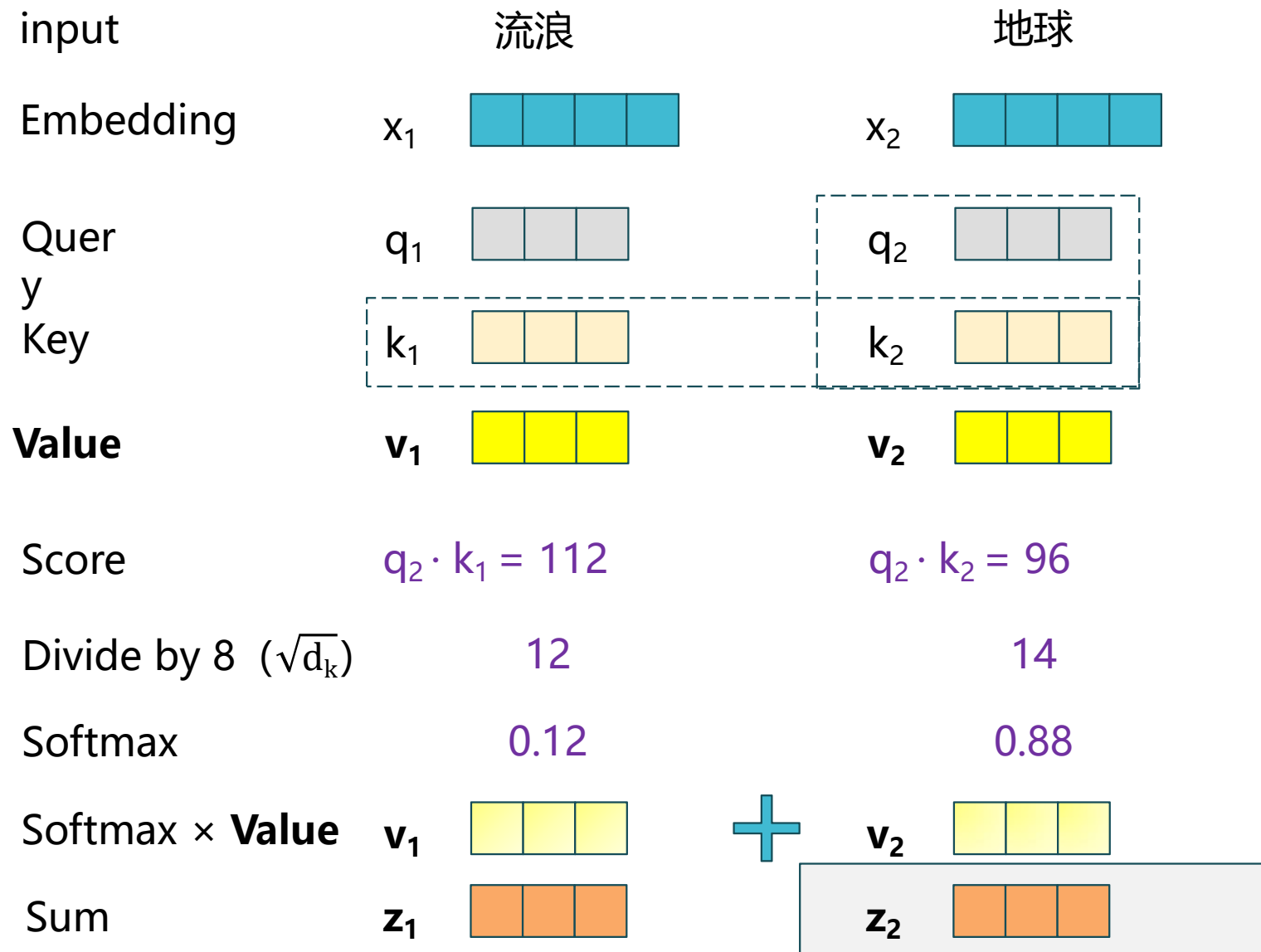
input	流浪	地球
Embedding	x_1 	x_2 
Query Key	q_1 	q_2 
	k_1 	k_2 
Value	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Softmax × Value	v_1 	v_2 
-----------------	---	---

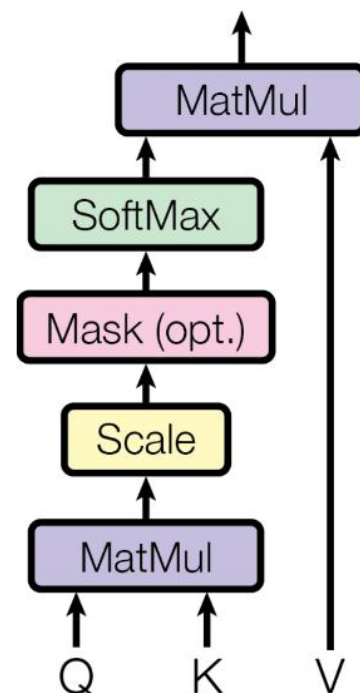
自注意力机制



自注意力机制



Scaled Dot-Product Attention

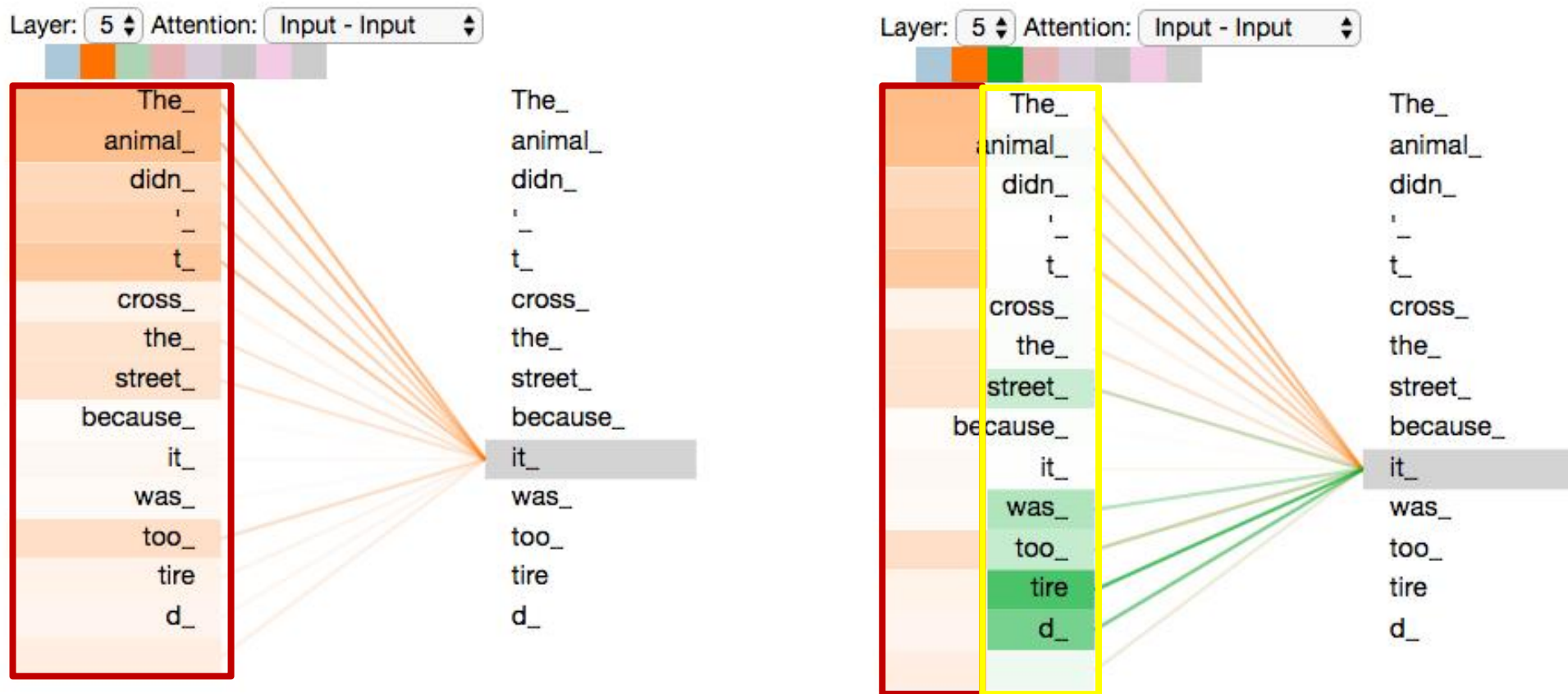


自注意力机制

$$\text{softmax} \left(\frac{q \times k^T}{\sqrt{d_k}} \right) v = \begin{matrix} z_1 \\ z_2 \end{matrix} \begin{bmatrix} \text{orange} & \text{orange} & \text{orange} \\ \text{orange} & \text{orange} & \text{orange} \end{bmatrix}$$

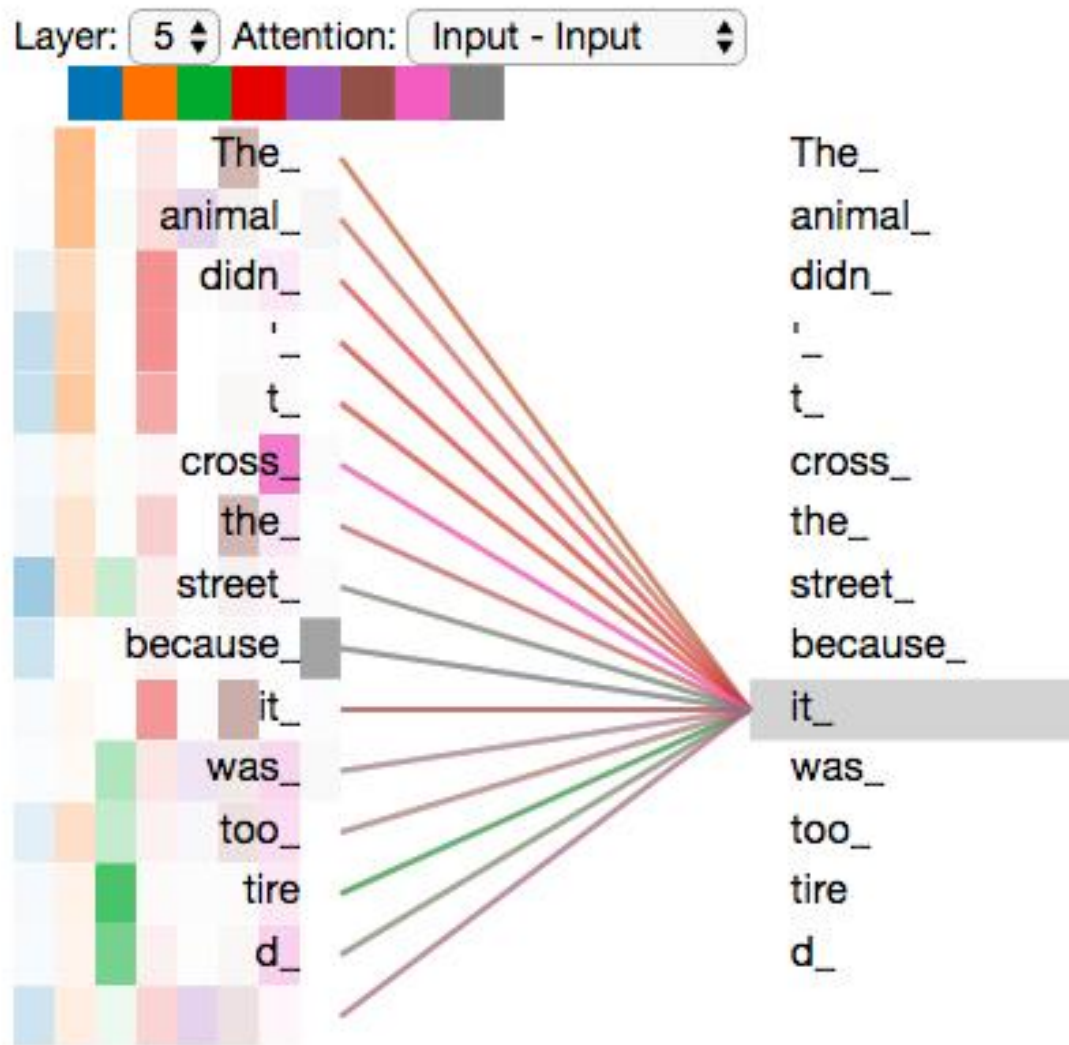
The diagram illustrates the self-attention mechanism. It shows a query matrix q (2x3, light blue) multiplied by a key matrix k^T (3x2, light orange). The result is divided by $\sqrt{d_k}$ and passed through a softmax function. This is then multiplied by a value matrix v (2x3, yellow) to produce the final output matrix, which is a 2x3 grid of orange cells. The output is labeled with z_1 and z_2 on the left.

多头注意力



每个attention可以关注不同的部分

多头注意力



多头注意力

input

流浪

attention1

attention2

Embedding

x_1



Query

q_{11}

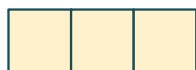


q_{12}

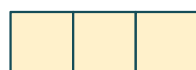


Key

k_{11}

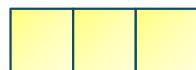


k_{12}

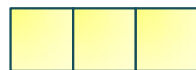


Value

v_{11}

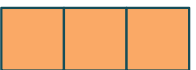


v_{12}

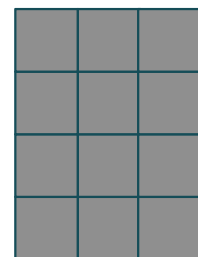


Sum

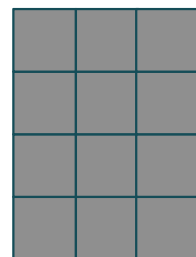
z_{11}



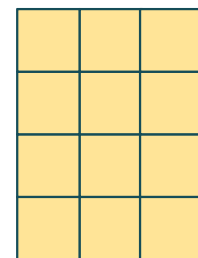
z_{12}



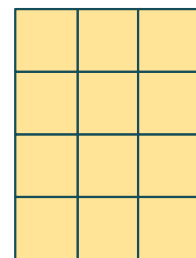
W_1^Q



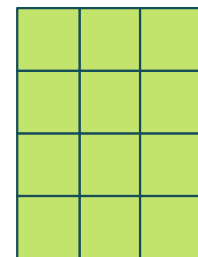
W_2^Q



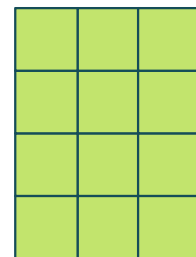
W_1^K



W_2^K

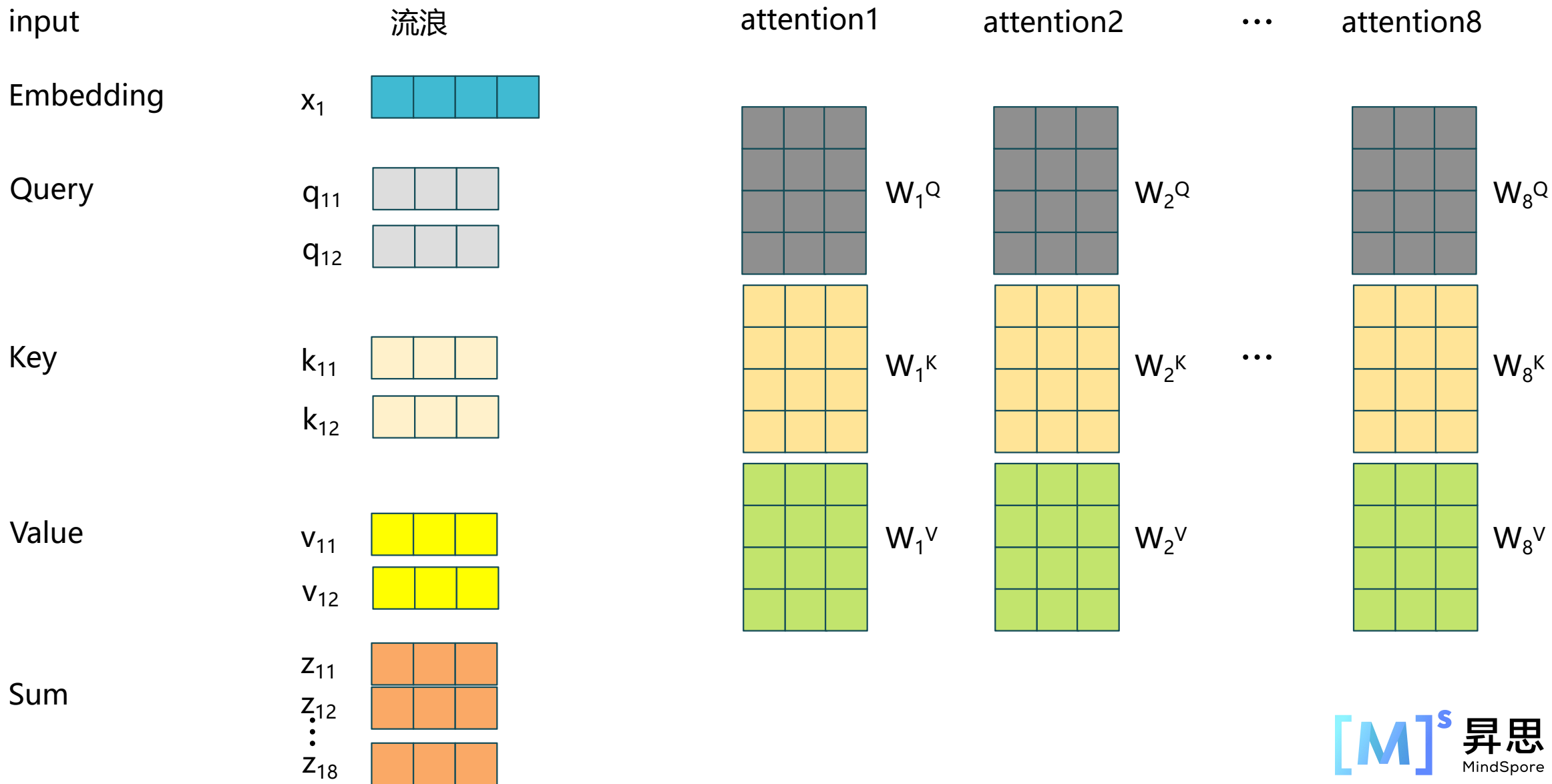


W_1^V



W_2^V

多头注意力



多头注意力

input

流浪

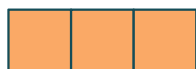
Embedding

x_1



Sum

z_{11}

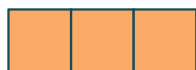


z_{12}

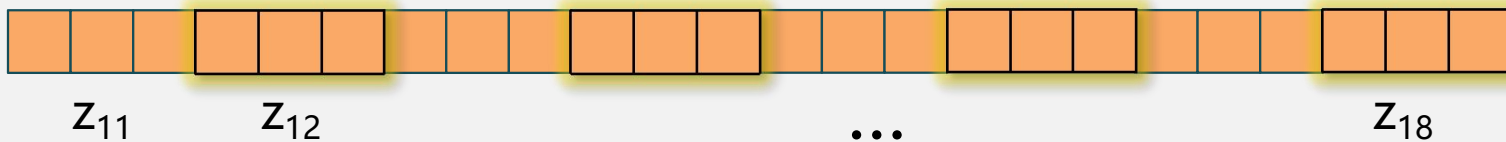


$\vdots$

z_{18}



Concat



多头注意力

input

流浪

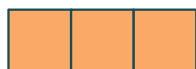
Embedding

x_1



Sum

z_{11}

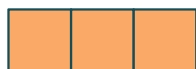


z_{12}

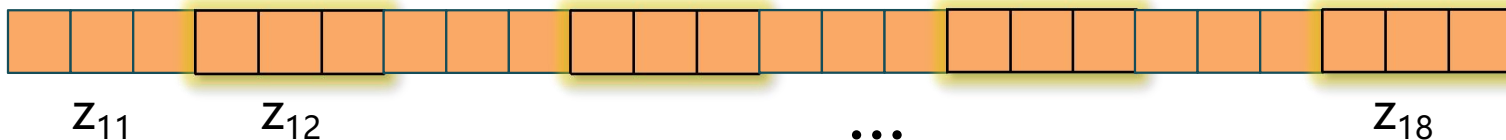


$\vdots$

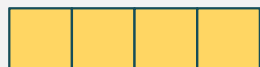
z_{18}



Concat

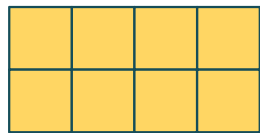


Concat $\times W^1$

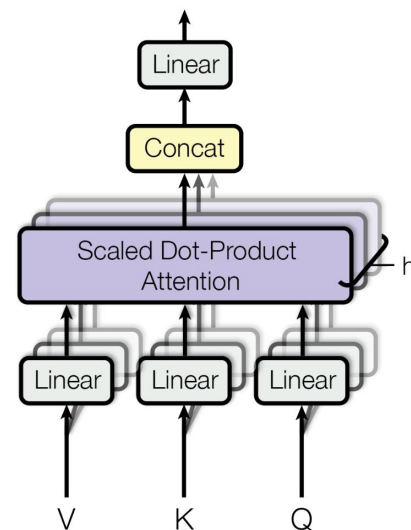


z

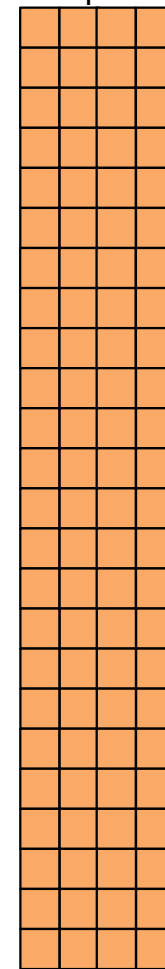
流浪
地球



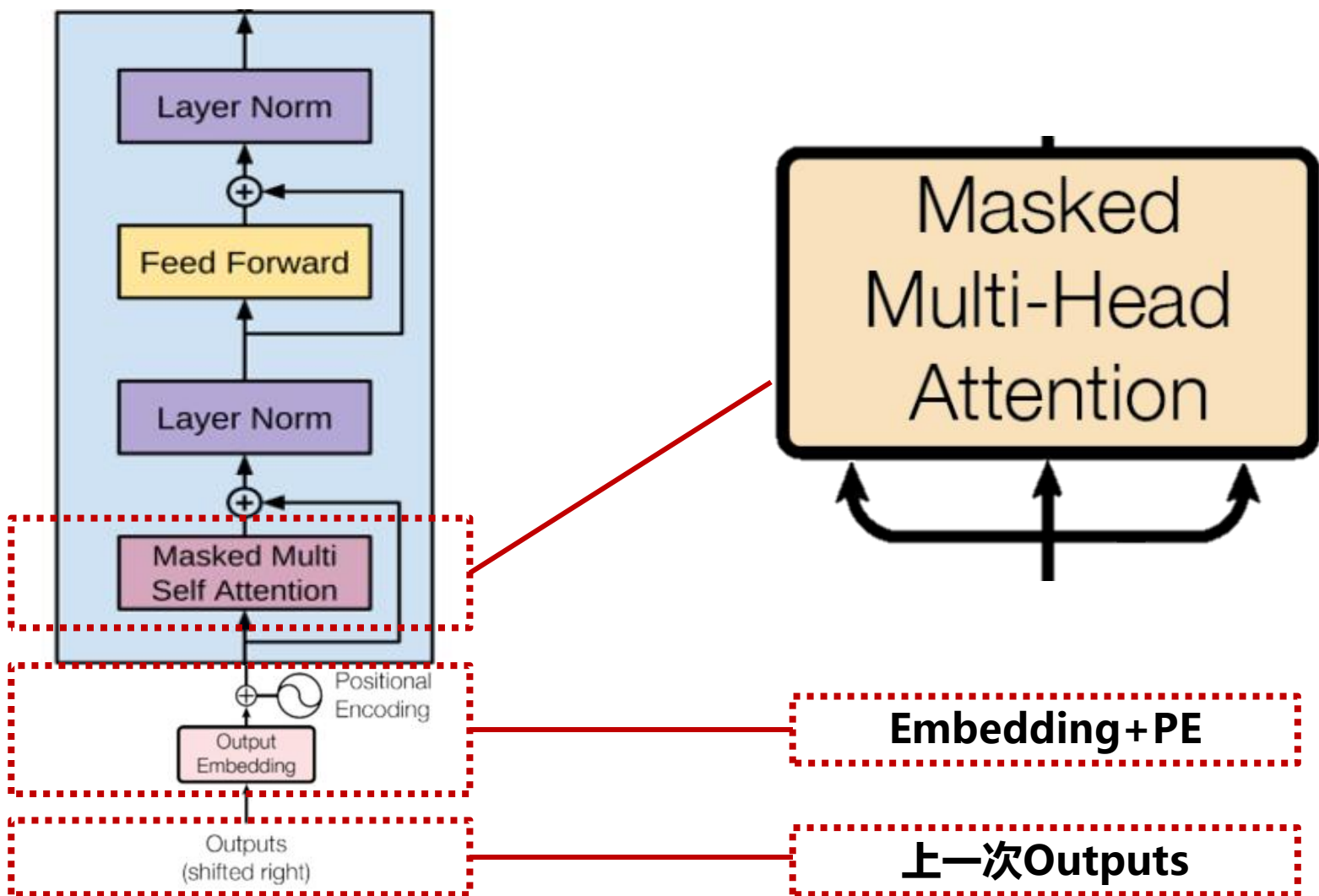
Multi-Head Attention



W_1



掩码多头注意力

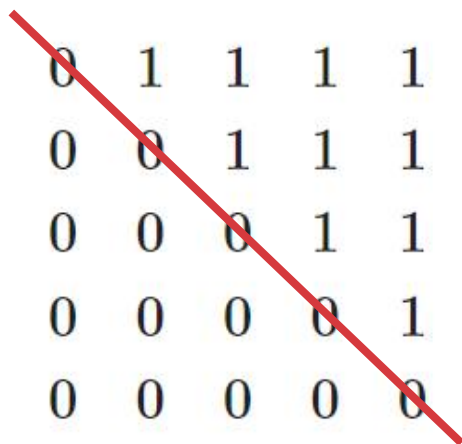


Shifted right 右移一位：句子开始需要有一个“开始标识符”

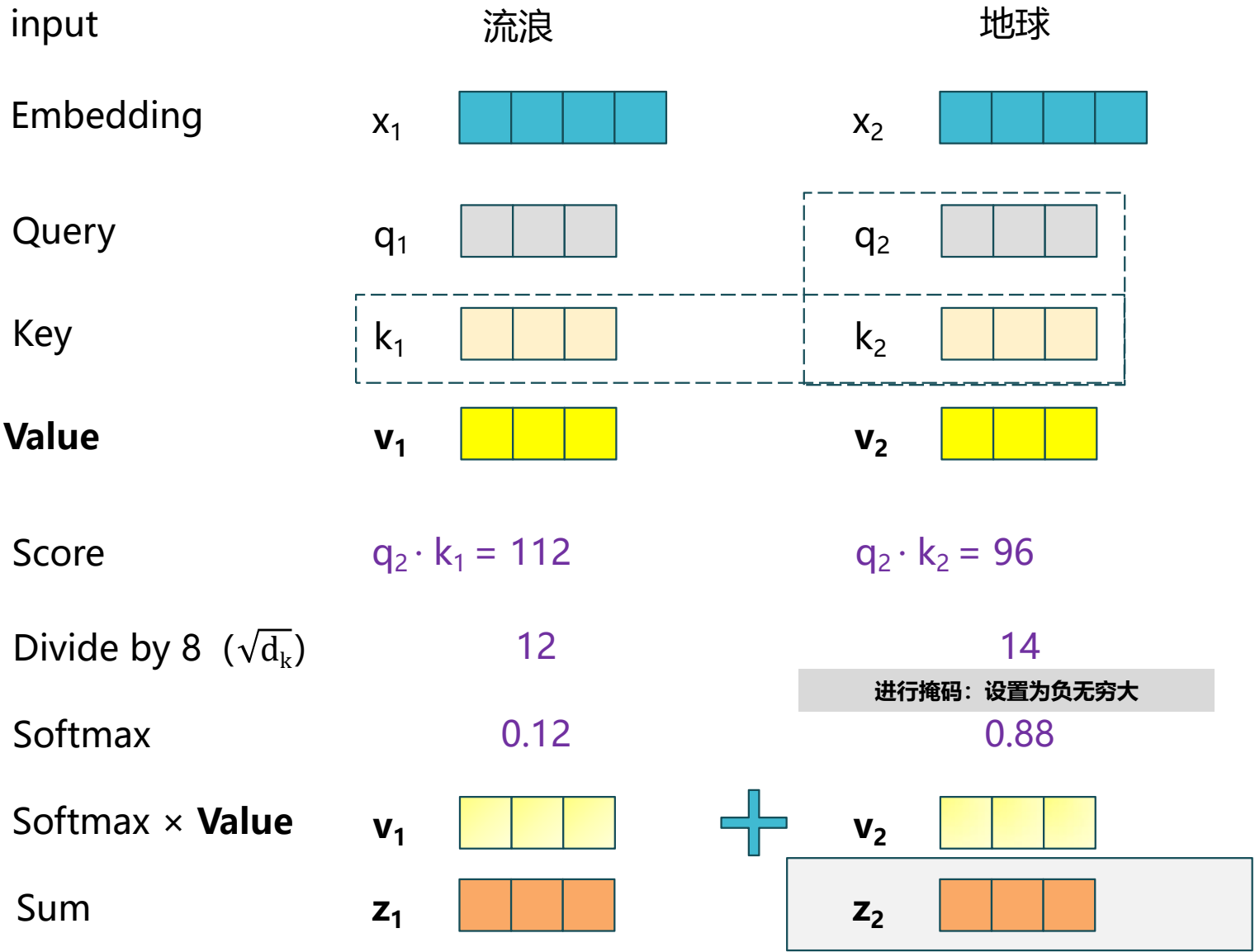
掩码多头注意力

在处理目标序列的输入时， t 时刻的模型只能“观察”直到 $t-1$ 时刻的所有词元，后续的词元不应该一并输入Decoder中。

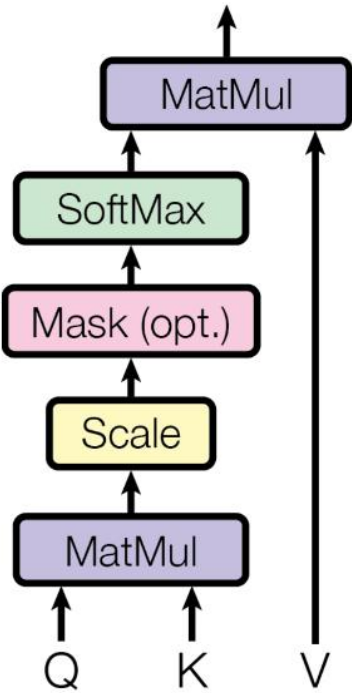
为了保证在 t 时刻，只有 $t-1$ 个词元作为输入参与多头注意力分数的计算，我们需要在第一个多头注意力中额外增加一个掩码，使目标序列中的词随时间发展逐个被暴露出来。该掩码一般被称作 subsequent mask，可通过三角矩阵实现，对角线以上的词元表示为不参与注意力计算的词元，标记为1。在计算注意力分数时，将标记为1的词元注意力分数设置成负无穷大，以便于让其在通过 softmax之后变成0



掩码多头注意力



Scaled Dot-Product Attention



minGPT中Attention代码实现 (详见model.py)

```
class CausalSelfAttention(nn.Cell):
```

```
    """
```

```
    一个带有末尾投影的纯多头掩码自注意力层。
```

```
    """
```

```
def __init__(self, config):
```

```
    super().__init__()
```

```
    assert config.n_embd % config.n_head == 0
```

```
    # 批处理所有头的key, query, value和projections
```

```
    self.c_attn = nn.Dense(config.n_embd, 3 * config.n_embd)
```

```
    # 输出投影
```

```
    self.c_proj = nn.Dense(config.n_embd, config.n_embd)
```

```
    # 正则化
```

```
    self.attn_dropout = nn.Dropout(p=config.attn_pdrop)
```

```
    self.resid_dropout = nn.Dropout(p=config.resid_pdrop)
```

```
    # 因果掩码, 用于确保注意力仅作用于输入序列的左侧部分
```

```
    self.bias = mint.tril(
```

```
        mint.ones((config.block_size, config.block_size), dtype=ms.int32)
    ).view(1, 1, config.block_size, config.block_size)
```

```
    self.n_head = config.n_head
```

```
    self.n_embd = config.n_embd
```

QKV映射, 往往为了简化计算量, 采取hidden states一次性映射+split为Q、K、V, 而非分别实例化三个全连接层

输出映射

构建因果掩码 (三角矩阵)

minGPT中Attention代码实现 (详见model.py)

```
def construct(self, x):  
    B, T, C = (  
        x.shape  
    ) # batch size, sequence length, embedding dimensionality (n_embd)
```

```
# 计算批次中所有头的query, key, values, 并将头向前移动以使其成为批次维度  
q, k, v = self.c_attn(x).split(self.n_embd, dim=2)
```

```
k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)  
q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)  
v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
```

```
# 因果自注意力: 自注意力: (B, nh, T, hs) 与 (B, nh, hs, T) 相乘 -> (B, nh, T, T)
```

```
att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.shape[-1]))  
att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float("-inf"))  
att = manual_softmax(att, dim=-1)  
att = self.attn_dropout(att)
```

```
y = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)  
y = y.transpose(1, 2).view(B, T, C)
```

```
y = self.resid_dropout(self.c_proj(y))
```

```
return y
```

$(B, T, C) \rightarrow (B, T, 3 \cdot n_{\text{embed}})$
 $\rightarrow (B, T, n_{\text{embed}}) \cdot 3$

多头注意里的并行计算:
 $(B, T, n_{\text{embed}}) \rightarrow$
 $(B, T, n_{\text{head}}, \text{head_size}) \rightarrow$
 $(B, n_{\text{head}}, T, \text{head_size},)$

$\rightarrow \text{softmax}(QK^T/\text{scaling})V$

$\rightarrow$ 输出映射

目录

01 minGPT介绍

02 Attention机制实现

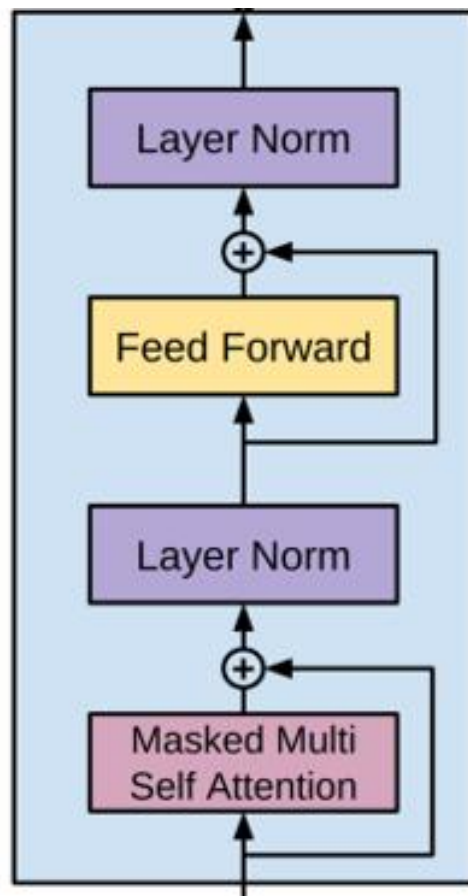
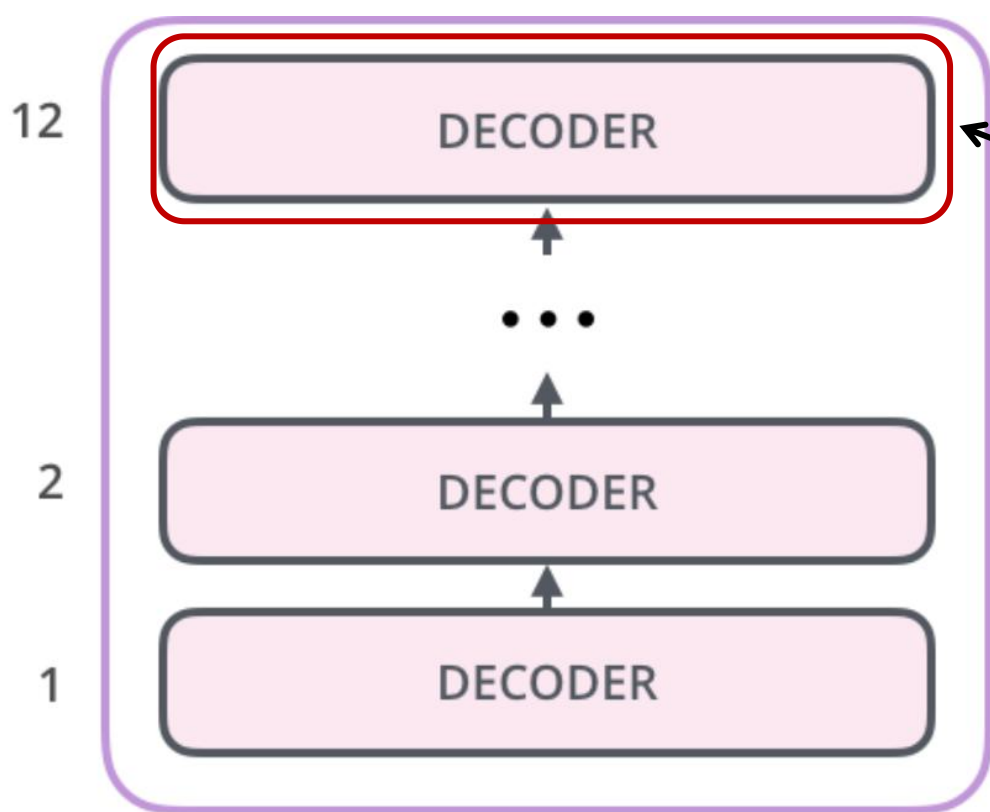
03 GPT架构

04 解码原理

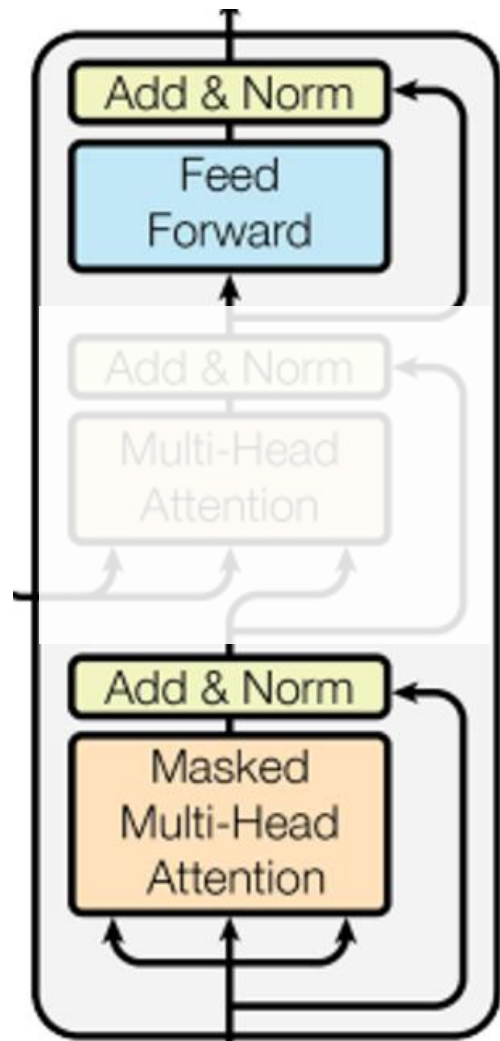
05 混合精度

GPT1模型结构

GPT1由Decoder Layer堆叠而成，Decoder Layer组成与Transformer Decoder Layer相似，只不过没有了计算encoder输出和decoder输入之间注意力分数的multi-head attention。



VS

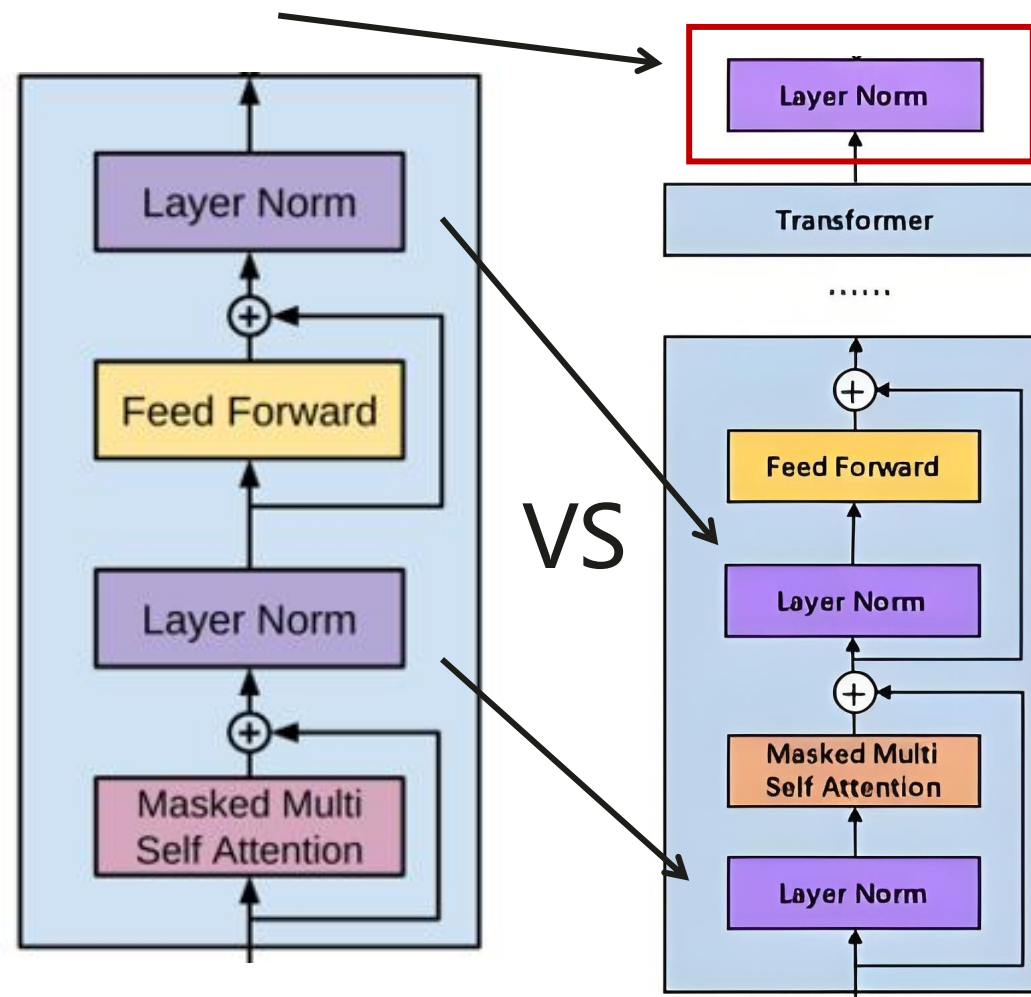


Reference: The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning)

GPT2模型结构

GPT2在GPT1的基础上修改了部分结构，具体表现为：

1. Attention计算前先进行Layer Norm，随后计算残差值。
2. Feed Forward前进行Layer Norm，随后计算残差值。
3. 若干个block后额外做一次Layer Norm处理后，在分发给下游任务。



minGPT中GPT2的模型配置

minGPT内部集成了多种GPT模型供开发者选择。此外，开发者还可以在该字典中添加自己的GPT模型配置。

其中：

- `n_layer`参数表示block数量
- `n_head`参数表示多头注意力头数
- `n_embd`表示输入序列长度

```
# 将“模型类型”转换为“详细配置”
config.merge_from_dict(
    {
        # 名称遵循 HuggingFace 的命名规范
        # GPT-1
        "openai-gpt": dict(n_layer=12, n_head=12, n_embd=768), # 117M params
        # GPT-2 configs
        "gpt2": dict(n_layer=12, n_head=12, n_embd=768), # 124M params
        "gpt2-medium": dict(n_layer=24, n_head=16, n_embd=1024), # 350M params
        "gpt2-large": dict(n_layer=36, n_head=20, n_embd=1280), # 774M params
        "gpt2-xl": dict(n_layer=48, n_head=25, n_embd=1600), # 1558M params
        # Gophers
        "gopher-44m": dict(n_layer=8, n_head=16, n_embd=512),
        "gpt-mini": dict(n_layer=6, n_head=6, n_embd=192),
        "gpt-micro": dict(n_layer=4, n_head=4, n_embd=128),
        "gpt-nano": dict(n_layer=3, n_head=3, n_embd=48),
    }[config.model_type]
)
```

目录

01 minGPT介绍

02 Attention机制实现

03 GPT架构

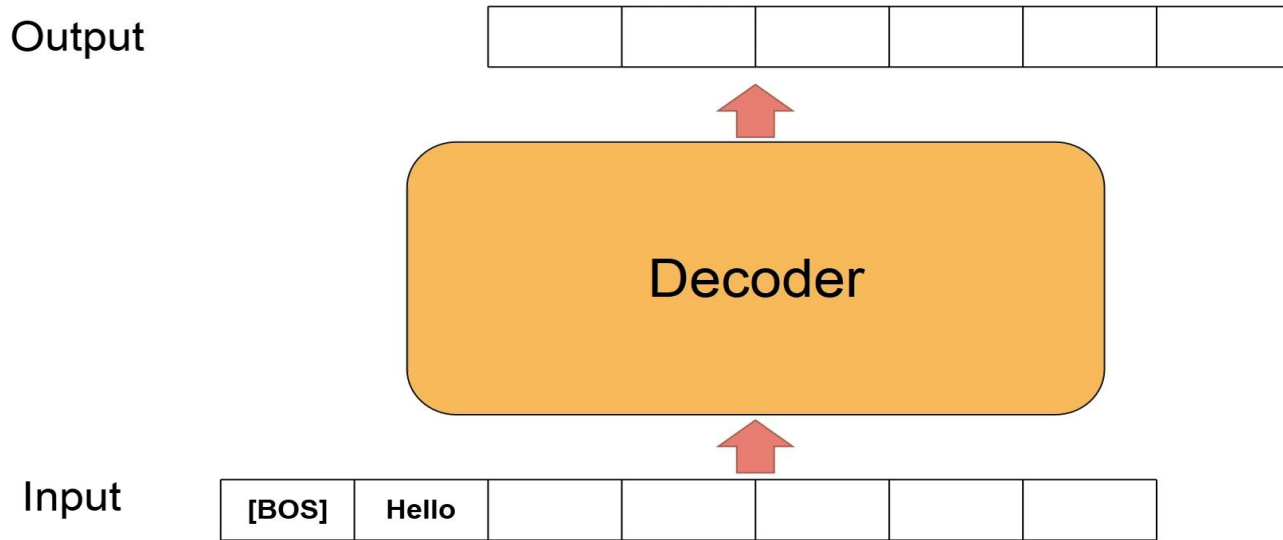
04 解码原理

05 混合精度

文本生成解码

在自然语言处理（NLP）的任务中，模型的原始输出通常是一种概率分布，也就是在词汇表的每个词的概率分布，而我们通常看到的输出是一个句子（系列单词），这个变换过程就是文本生成的解码。

这个过程其实是选择一个最优序列，使得输出序列的条件概率最大，常用方法有贪心搜索（Greedy Search）、束搜索（Beam Search）等，如下图是一个基于贪心搜索的GPT文本生成。



贪心搜索

贪心搜索，即每一个时间步都取出一个条件概率最大的输出，再将从开始到当前步的结果作为输入去获得下一个时间步的输出，直到模型给出生成结束的标志。

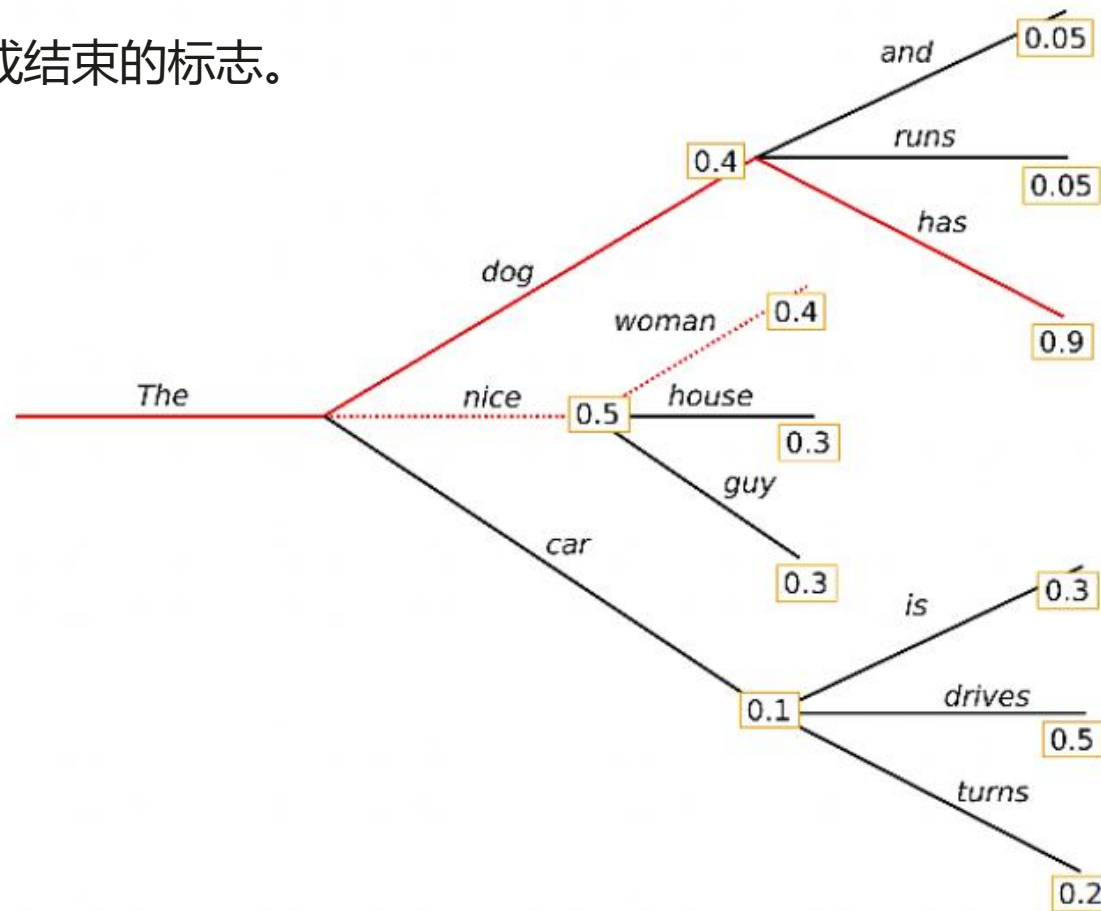
不过**当前的最优不一定全局的最优**。如右图：

按照贪心搜索输出序列的条件概率为：

$$0.5 \times 0.4 = 0.2$$

其实最优的输出序列的条件概率为：

$$0.4 \times 0.9 = 0.36$$



Temperature采样 (Temperature Sampling)

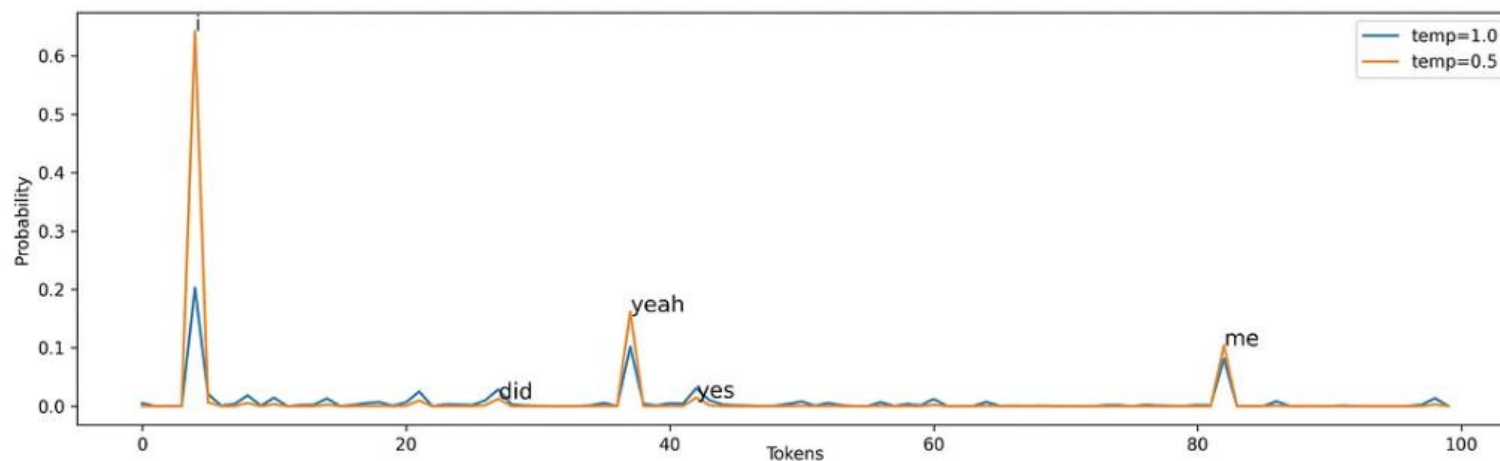
Temperature Sampling通过设置参数temperature，控制调整token之间的概率分布。

temperature的取值范围一般是 $0 < temp \leq 1$ ，随着temp降低，原本概率高的token对应概率更高，原本概率低的token对应概率更低，模型更具有确定性。

Temperature Sampling计算公式

(温度: t)

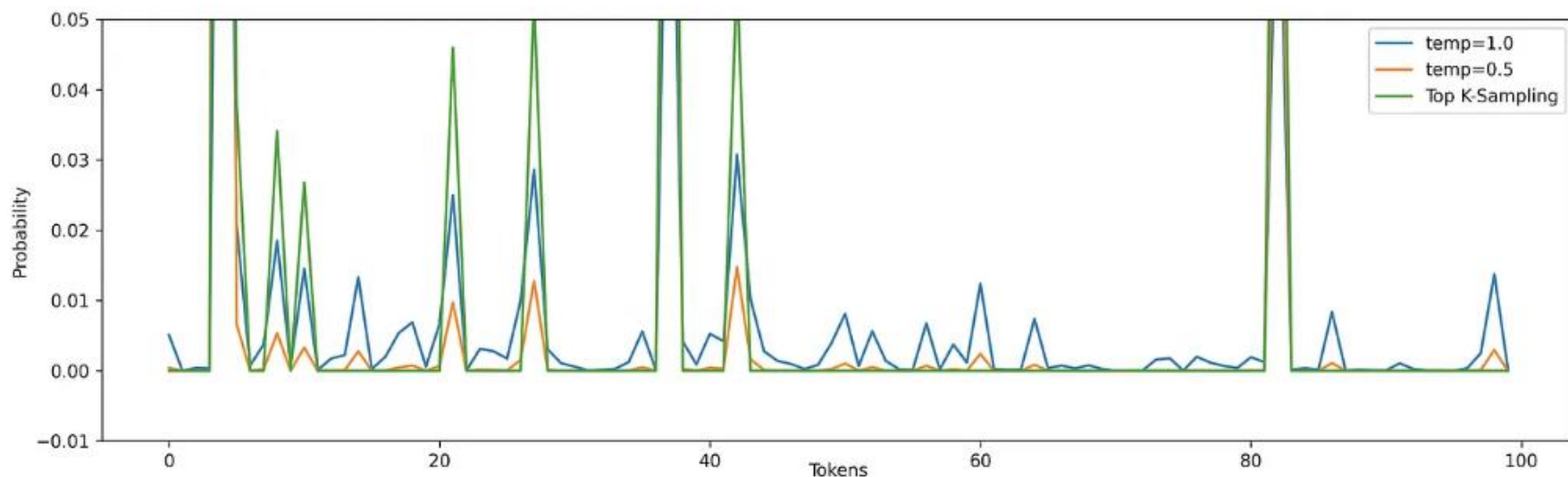
$$P(x_i|x_{1:i-1}) = \frac{\exp(u_i/t)}{\sum_j \exp(u_j/t)}$$



TopK采样 (TopK Sampling)

Top-K采样是每个时间步只从概率最高的前K个单词中进行随机采样，而不考虑其他低概率的单词，即这些单词的采样概率为0。这样可以避免采样到一些不合适或者不相关的token，同时可以保留一些有趣或有创意的词汇。

Top-K采样和Temperature采样相比模型更加确定，概率小的token根本不会被采样到。缺陷是无法确定k取值是什么时得到的输出序列是最优的。



联合采样

```
def generate(self, idx, max_new_tokens, temperature=1.0, do_sample=False, top_k=None):  
    """
```

```
# 在最后一步提取logits, 并根据所需的温度进行缩放
```

```
logits = logits[:, -1, :] / temperature
```

```
# (可选地) 将对数输出值裁剪为仅前 k 个选项的值
```

```
if top_k is not None:
```

```
    v, _ = mint.topk(logits, top_k)
```

```
    logits[logits < v[:, [-1]]] = -float("Inf")
```

```
# 应用 softmax 函数将对数输出转换为 (归一化的) 概率值
```

```
probs = mint.nn.functional.softmax(logits, dim=-1)
```

```
# 要么从该分布中抽取一个样本, 要么选取最有可能的元素
```

```
if do_sample:
```

```
    idx_next = mint.multinomial(probs, num_samples=1)
```

```
else:
```

```
    _, idx_next = mint.topk(probs, k=1, dim=-1)
```

```
# 将采样的索引添加到运行序列中, 并继续进行
```

在minGPT中, 我们先进行了temperature采样 (默认值为1, 可自行修改), 随后提供给开发者greedy search和Top-K采样两种方式, 开发者可以按照自己喜好进行调用。

任务一：minGPT模型实现（完成model.py代码补全）

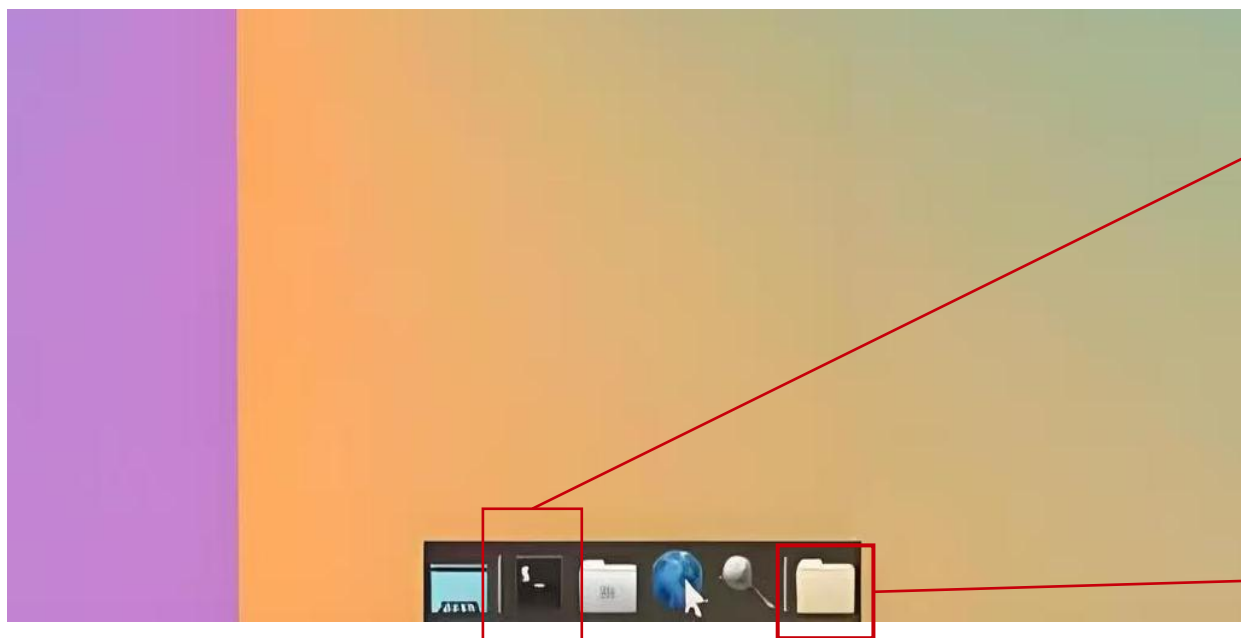
经过Attention机制实现、GPT架构、解码原理三部分的背景知识补充，请开发者完成minGPT实验指导手册中的任务一部分。

香橙派Alpro登录



输入密码：
Mind@123

香橙派AIpro页面展示



单击即可打开终端，后续
运行代码可在终端操作

单击即可打开文件资源管
理器，选择minGPT-
Open Folder即可进入本
课程文件夹

目录

01 minGPT介绍

02 Attention机制实现

03 GPT架构

04 解码原理

05 混合精度

自动混合精度介绍

混合精度 (Mix Precision) 训练是指在训练时，对神经网络不同的运算采用不同的数值精度的运算策略。在神经网络运算中，部分运算对数值精度不敏感，此时使用较低精度可以达到明显的加速效果（如conv、matmul等）；而部分运算由于输入和输出的数值差异大，通常需要保留较高精度以保证结果的正确性（如log、softmax等）。

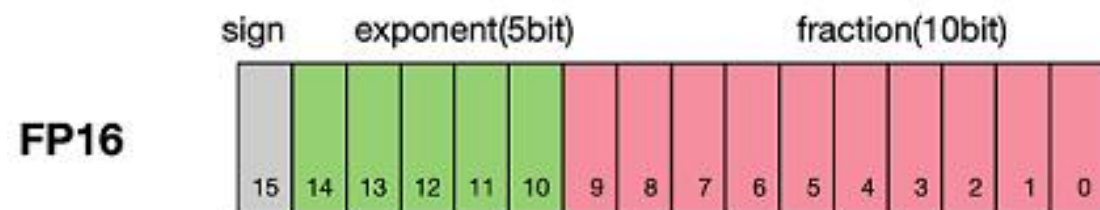
当前的AI加速卡通常通过针对计算密集、精度不敏感的运算设计了硬件加速模块，如Ascend NPU的Cube等。对于conv、matmul等运算占比较大的神经网络，其训练速度通常会有较大的加速比。

MindSpore内置的amp模块提供了便捷的自动混合精度接口，用户可以在不同的硬件后端通过简单的接口调用获得训练加速。下面我们对混合精度计算原理进行简介，而后通过实例介绍MindSpore的自动混合精度用法。

混合精度计算原理

浮点数据类型主要分为双精度（FP64）、单精度（FP32）、半精度（FP16）。在神经网络模型的训练过程中，一般默认采用单精度（FP32）浮点数据类型，来表示网络模型权重和其他参数。在了解混合精度训练之前，这里简单了解浮点数据类型。

根据IEEE二进制浮点数算术标准（[IEEE 754](#)）的定义，浮点数据类型分为双精度（FP64）、单精度（FP32）、半精度（FP16）三种，其中每一种都有三个不同的部分来表示。FP64表示采用8个字节共64位，来进行的编码存储的一种数据类型；同理，FP32表示采用4个字节共32位来表示；FP16则是采用2个字节共16位来表示。如图所示：



FP16：计算速度快，表示范围有限

FP32：计算速度慢，精度高



混合精度计算原理

从图中可以看出，与FP32相比，FP16的存储空间是FP32的一半。类似地，FP32则是FP64的一半。因此使用FP16进行运算具备以下优势：

1. **减少内存占用**：FP16的位宽是FP32的一半，因此权重等参数所占用的内存也是原来的一半，节省下来的内存可以放更大的网络模型或者使用更多的数据进行训练。
2. **计算效率更高**：在特殊的AI加速硬件下，使用FP16的执行运算性能比FP32更加快。

但是使用FP16同样会带来一些问题：

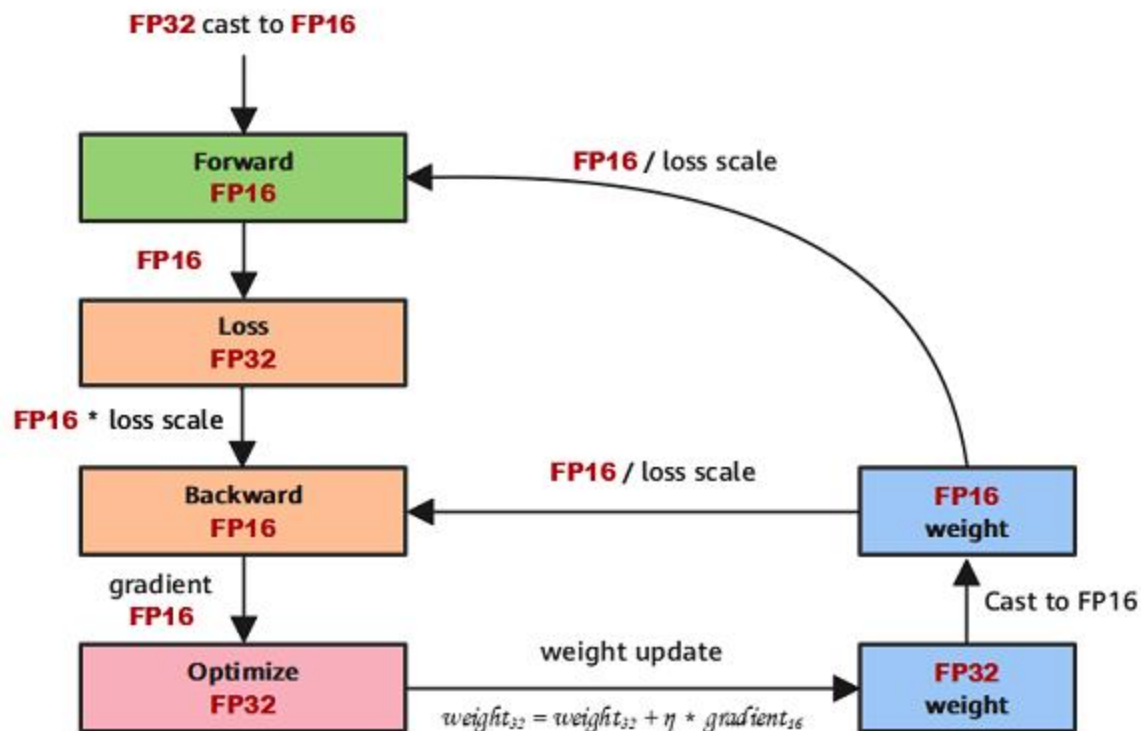
1. **数据溢出**：FP16的有效数据表示范围为 $[5.9 \times 10^{-8}, 65504]$ ，FP32的有效数据表示范围为 $[1.4 \times 10^{-45}, 1.7 \times 10^{38}]$ 。可见FP16相比FP32的有效范围要窄很多，使用FP16替换FP32会出现上溢（Overflow）和下溢（Underflow）的情况。而在深度学习中，需要计算网络模型中权重的梯度（一阶导数），因此梯度会比权重值更加小，往往容易出现下溢情况。
2. **舍入误差**：Rounding Error是指当网络模型的反向梯度很小，一般FP32能够表示，但是转换到FP16会小于当前区间内的最小间隔，会导致数据溢出。如0.00006666666在FP32中能正常表示，转换到FP16后会表示成为0.000067，不满足FP16最小间隔的数会强制舍入。

混合精度计算原理

因此，在使用混合精度获得训练加速和内存节省的同时，需要考虑FP16引入问题的解决。Loss Scale损失缩放，FP16类型数据下溢问题的解决方案，其主要思想是在计算损失值loss的时候，将loss扩大一定的倍数。根据链式法则，梯度也会相应扩大，然后在优化器更新权重时再缩小相应的倍数，从而避免了数据下溢。

整体流程见右图，总结下来为7步，红色部分需要人为设置

1. 参数以FP32存储；
2. 正向计算过程中，遇到FP16算子，需要把算子输入和参数从FP32 cast成FP16进行计算；
3. 将Loss层设置为FP32进行计算；
4. 反向计算过程中，首先乘以Loss Scale值，避免反向梯度过小而产生下溢；
5. FP16参数参与梯度计算，其结果将被cast回FP32；
6. 除以Loss scale值，还原被放大的梯度；
7. 判断梯度是否存在溢出，如果溢出则跳过更新，否则优化器以FP32对原始参数进行更新。



MindSpore的自动类型转换

mindspore.amp.auto_mixed_precision接口提供对网络做自动类型转换的功能。自动类型转换遵循黑白名单机制，根据常用的运算精度习惯配置了5个等级，分别为：

```
mindspore.amp.auto_mixed_precision(network, amp_level='O0', dtype=mstype.float16) \[源代码\]
```

- 'O0'：神经网络保持FP32；
- 'O1'：按白名单将运算cast为FP16；
- 'O2'：按黑名单（BatchNorm、LayerNorm）保留FP32，其余运算cast为FP16；
- 'O3'：神经网络完全cast为FP16。
- 'auto'：白名单内的算子运算cast为FP16，黑名单内的算子运算cast为FP32，其余算子选择算子输入的最高浮点精度来转换。

需要注意的是：当前黑白名单为Cell粒度

设置为O2混合精度模式

昇腾开发板（香橙派）为了高性能，训练架构设计之初便是基于float16，因此在香橙派上训练，需要手动设置为O2混合精度模式。

具体操作非常简单，模型实例化后使用amp.auto_mixed_precision接口重新封装模型即可。

```
from mindspore import amp

# 香橙派训练必须使用设置o2模式的混合精度
model = amp.auto_mixed_precision(model, 'o2')
model.set_train()
```

Loss放大，乘以Loss Scale值

香橙派上目前缺失
amp模块中
LossScaler类所需的
ops.select方法算子，
需要继承LossScaler类
后手动实现。
以
DynamicLossScaler
为例。

```
# 香橙派上没有ops.select方法的算子，需要手动实现后继承amp.DynamicLossScaler，对其adjust方法进行修改
def select(condition, input, other):
    return condition * input + (~condition) * other

class CustomDynamicLossScaler(amp.DynamicLossScaler):

    def __init__(self, scale_value, scale_factor, scale_window):
        super().__init__(scale_value, scale_factor, scale_window)

    def adjust(self, grads_finite):
        """
        使用自定义的select方法而不是ops.select调整`scale_value`。

        参数:
        | grads_finite (Tensor): 一个标量bool张量，表示梯度是否有限。
        """
        one = ops.ones((), self.scale_value.dtype)
        scale_mul_factor = self.scale_value * self.scale_factor

        scale_value = select(grads_finite,
                             select(self.counter==(self.scale_window-1),
                                     select(ops.isfinite(scale_mul_factor),
                                             scale_mul_factor, self.scale_value), self.scale_value),
                             ops.maximum(one, self.scale_value / self.scale_factor))
        ops.assign(self.scale_value, scale_value)
        counter = ((self.counter + 1) % self.scale_window) * grads_finite

        ops.assign(self.counter, counter)

    return True
```

Loss放大，乘以Loss Scale值

之后实例化自己定义的LossScaler，并在net_forward函数中调整返回的loss值

```
loss_scaler = CustomDynamicLossScaler(scale_value=2**16, scale_factor=2, scale_window=50)

def net_forward(x, y):
    logits = model(x)
    logits = logits.float()
    loss = mint.nn.functional.cross_entropy(logits.view(-1, logits.shape[-1]), y.view(-1), ignore_index=-1)

    # 动态调整loss值
    return loss_scaler.scale(loss)
```

还原Loss

```
loss, grads = grad_fn(x, y)
# 把放大后的loss缩小回原始数值
self.loss = loss_scaler.unscale(loss)
```

溢出检测，缩小梯度，模型更新

```
# 判断是否有溢出
is_finite = amp.all_finite(grads)
if is_finite:
    unscaled_grads = loss_scaler.unscale(grads)
    unscaled_grads = clip_by_norm(unscaled_grads, config.grad_norm_clip) # 梯度裁剪
    self.optimizer(unscaled_grads)
# 动态调整LossScaler数值
loss_scaler.adjust(is_finite)
```

溢出检测

缩小梯度

LossScaler自调整

关于Loss Scaler

MindSpore.amp中LossScaler主要有两种：

- DynamicLossScaler：根据梯度溢出情况调整scale_value
- StaticLossScaler：固定scale_value

主要使用以下方法：

- scale: 根据scale_value缩放值
- unscale: 根据scale_value还原值
- ajust: 调整scale_value的值（仅限DynamicLossScaler 使用）

任务二：香橙派上的O2混合精度训练实现（完成trainer.py代码补全）

经过混合精度部分的背景知识补充，请开发者完成minGPT实验指导手册中的任务二部分。

实验验证

完成项目代码填空后，请按照以下步骤进行实验验证

步骤 1 按照实验指导手册完成 model.py 和 trainer.py 的代码补充。

步骤 2 打开终端进入 minGTP 文件夹中。

```
(base) HwHiAiUser@orangepiaipro-20t:~$ cd minGTP/
```

步骤 3 检查文件结构。

```
(base) HwHiAiUser@orangepiaipro-20t:~/minGTP$ ls
demo.py  mingpt  run_demo.sh
(base) HwHiAiUser@orangepiaipro-20t:~/minGTP$ cd mingpt/
(base) HwHiAiUser@orangepiaipro-20t:~/minGTP/mingpt$ ls
__init__.py  model.py  trainer.py  utils.py
(base) HwHiAiUser@orangepiaipro-20t:~/minGTP/mingpt$
```

步骤 4 回到 minGTP 文件目录，使用 bash 命令运行 run_demo.sh 脚本

```
(base) HwHiAiUser@orangepiaipro-20t:~/minGTP/mingpt$ cd ..
(base) HwHiAiUser@orangepiaipro-20t:~/minGTP$ bash run_demo.sh
```

预期输出：

```
Int32 Int32
[MS_DEV_RUNTIME_CONF]Runtime config: all_finite:false
[MS_RUNTIME_PROF]The jit level is: 00, and enable kernelbykernel executor in the PYNATIVE mode.

1 -1
0 -1
1 -1
2 -1
2 -1
1 0
0 1
1 1
1 1
1 2
2 2

number of parameters: 0.09M
graph kernel flags = ""
..iter_dt 0.00ms; iter 0: train loss 1.09329
iter_dt 381.06ms; iter 100: train loss 0.15819
iter_dt 358.79ms; iter 200: train loss 0.06385
iter_dt 356.11ms; iter 300: train loss 0.03205
iter_dt 329.58ms; iter 400: train loss 0.03756
iter_dt 373.87ms; iter 500: train loss 0.03541
iter_dt 357.78ms; iter 600: train loss 0.01390
iter_dt 367.16ms; iter 800: train loss 0.00134
iter_dt 373.33ms; iter 700: train loss 0.01619
iter_dt 365.10ms; iter 900: train loss 0.00827
iter_dt 349.74ms; iter 1000: train loss 0.02051
iter_dt 406.40ms; iter 1100: train loss 0.01116
iter_dt 359.97ms; iter 1200: train loss 0.00169
iter_dt 342.77ms; iter 1300: train loss 0.00879
iter_dt 332.45ms; iter 1400: train loss 0.00069
iter_dt 382.37ms; iter 1500: train loss 0.00095
iter_dt 351.18ms; iter 1600: train loss 0.02079
iter_dt 327.24ms; iter 1700: train loss 0.01234
iter_dt 363.84ms; iter 1800: train loss 0.03111
iter_dt 349.88ms; iter 1900: train loss 0.02852
train final score: 5000/5000 = 100.00% correct
GPT claims that [2, 2, 2, 2, 2, 0] sorted is [0, 1, 2, 2, 2, 2] but gt is [0, 2, 2, 2, 2, 2]
GPT claims that [2, 2, 2, 2, 2, 0] sorted is [0, 1, 2, 2, 2, 2] but gt is [0, 2, 2, 2, 2, 2]
GPT claims that [2, 2, 2, 2, 2, 0] sorted is [0, 1, 2, 2, 2, 2] but gt is [0, 2, 2, 2, 2, 2]
test final score: 4975/5000 = 99.50% correct
input sequence : [[0, 0, 2, 1, 0, 1]]
predicted sorted: [[0, 0, 0, 1, 1, 2]]
gt sort        : [0, 0, 0, 1, 1, 2]
matches        : True
```

混合精度信息

部分数据集

模型参数数量

训练loss，每100个iter打印一次

完成训练后的模型在训练集和测试集的表现

随机生成一条数据在训练后模型的验证结果

A child wearing a brown aviator helmet and goggles sits on the shoulder of a white, humanoid robot. The child is pointing their right index finger towards a large, glowing globe in the background. The globe features a semi-transparent world map overlay. The scene is set against a light blue sky with several bright, diagonal streaks of light. The robot's head is turned towards the child, and its right arm is visible, holding the child's leg.

Thank you